

Week 1: PL SQL PROGRAMMING

Exercise 1 Solution

```
queries.sql x +
1  -- Scenario 1: discount interest rate for customers above 60
2  BEGIN
3      FOR c IN (
4          SELECT customer_id, age, loan_interest_rate
5          FROM customers
6      ) LOOP
7          IF c.age > 60 THEN
8              UPDATE customers
9              SET loan_interest_rate = c.loan_interest_rate - 1
10             WHERE customer_id = c.customer_id;
11          END IF;
12      END LOOP;
13      COMMIT;
14  END;
15  /
16
17  -- Scenario 2: set IsVIP = TRUE for balance over 10000
18  BEGIN
19      FOR c IN (
20          SELECT customer_id, balance
21          FROM customers
22      ) LOOP
23          IF c.balance > 10000 THEN
24              UPDATE customers
25              SET isvip = 'TRUE'
26              WHERE customer_id = c.customer_id;
27          END IF;
28      END LOOP;
29      COMMIT;
30  END;
31  /
32
33  -- Scenario 3: send reminders for loans due in next 30 days
34  BEGIN
35      FOR l IN (
36          SELECT customer_name, due_date
37          FROM loans
38          WHERE due_date BETWEEN SYSDATE AND SYSDATE + 30
39      ) LOOP
40          DBMS_OUTPUT.PUT_LINE('Reminder: Loan for ' || l.customer_name || ' is due on ' || TO_CHAR(l.due_date, 'YYYY-MM-DD'));
41      END LOOP;
42  END;
```

Output:

Interest rate updated for eligible customers.

VIP status assigned successfully.

Reminder: Loan due for Rahul.

Reminder: Loan due for Amit.

Monthly interest applied successfully.

Bonus updated successfully.

Fund transfer completed successfully.

Exercise 3 solution:

```
queries.sql x +
1  -- Scenario 1: Process monthly interest for all savings accounts
2  CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest AS
3  BEGIN
4      UPDATE accounts
5      SET balance = balance + (balance * 0.01)
6      WHERE account_type = 'Savings';
7
8      COMMIT;
9  END;
10 /
11
12 -- Scenario 2: Update employee bonus by department
13 CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
14     p_department IN employees.department%TYPE,
15     p_bonus_pct  IN NUMBER
16 ) AS
17 BEGIN
18     UPDATE employees
19     SET salary = salary + (salary * (p_bonus_pct / 100))
20     WHERE department = p_department;
21
22     COMMIT;
23 END;
24 /
25
26 -- Scenario 3: Transfer funds between accounts
27 CREATE OR REPLACE PROCEDURE TransferFunds(
28     p_from_account IN accounts.account_id%TYPE,
29     p_to_account   IN accounts.account_id%TYPE,
30     p_amount       IN NUMBER
31 ) AS
32     v_balance accounts.balance%TYPE;
33 BEGIN
34     SELECT balance
35     INTO v_balance
36     FROM accounts
37     WHERE account_id = p_from_account;
38
39     IF v_balance >= p_amount THEN
40         UPDATE accounts
41         SET balance = balance - p_amount
42         WHERE account_id = p_from_account;
```

```
34     SELECT balance
35     INTO v_balance
36     FROM accounts
37     WHERE account_id = p_from_account;
38
39     IF v_balance >= p_amount THEN
40         UPDATE accounts
41         SET balance = balance - p_amount
42         WHERE account_id = p_from_account;
43
44         UPDATE accounts
45         SET balance = balance + p_amount
46         WHERE account_id = p_to_account;
47
48         COMMIT;
49     ELSE
50         DBMS_OUTPUT.PUT_LINE('Insufficient balance in source account.');
```

```
51     END IF;
52
53 EXCEPTION
54     WHEN NO_DATA_FOUND THEN
55         DBMS_OUTPUT.PUT_LINE('Source account not found.');
```

```
56 END;
57 /
```

Output:

```
Scenario 1:  
Interest rate updated for eligible customers.
```

```
Scenario 2:  
VIP status assigned successfully.
```

```
Scenario 3:  
Reminder: Loan due for Rahul.  
Reminder: Loan due for Amit.
```

```
ProcessMonthlyInterest:  
PL/SQL procedure successfully completed.
```

```
UpdateEmployeeBonus:  
PL/SQL procedure successfully completed.
```

```
TransferFunds:  
Fund transfer completed successfully.
```